CUADERNILLOS DE DESARROLLO DE SOFTWARE

02

Arquitectura y Diseño de Interfaces Proyecto

De la teoría a la práctica: los 7 trabajos prácticos, el proyecto modelo, las plantillas y la cadena completa de trazabilidad para tu proyecto integrador.



Arquitectura y Diseño de Interfaces · 3° año · Ciclo 2026

Tecnicatura Superior en Desarrollo de Software · IES 9-018 · Prof. Paulo Alvarez

Fuente primaria: `github.com/IES9018/proyecto-adi-2026`

Se lee junto con: Cuadernillo 01 — Teoría · Cuadernillo 03 — PP3

Índice

Cómo usar este cuadernillo

Mapa del proyecto

TP1 — De Vibe Coding a SDD

TP2 — Arquitectura Visible

TP3 — Interfaces que no Confunden

TP4 — Contratos antes que Código

TP5 — Estrategia Mobile

TP6 — Integración Continua

TP Final — Defensa

Proyecto Modelo y Plantillas

Cadena de trazabilidad completa

Cómo usar este cuadernillo

> **Propósito:** que sepas qué producir en cada TP, con qué plantilla, cómo se conecta con los anteriores y con la teoría, y cómo cada artefacto alimenta el proyecto integrador.

Fuentes de esta colección: `IES9018/ADI-teoria-y-recursos` (teoría), `IES9018/proyecto-adi-2026` (proyecto), `IES9018/proyecto-pp3-2026` (coordinación).

Qué hace cada cuadernillo

Cuadernillo	Rol
01 — Teoría	Comprendés los conceptos
02 — Proyecto (este)	Aplicás: producís artefactos en cada TP
03 — PP3	Coordinás sprints, auditoría y defensa

> RECORDÁ
>
> **Este cuadernillo no reemplaza las consignas.** Cada TP tiene su README completo en el repositorio `proyecto-adi-2026`. Acá encontrás el mapa de navegación: qué se espera, cómo se conecta y qué artefacto dejar listo para el siguiente paso.

Cómo leer cada unidad

Cada TP sigue esta estructura:

Sección	Te dice
Qué producís	Artefacto concreto que deja el TP
Entrada	Qué necesitás tener antes de empezar
Teoría	Qué unidad del Cuadernillo 01 aplicar
Plantilla	Archivo del que partir
Siguiente	Qué consume este resultado

Archivos canónicos

Todos los TPs producen archivos dentro de tu repositorio individual:

```
tu-repo/
├── SPEC.md              ← evoluciona v1 → vFinal
├── .opencoderules        ← arnés v1 → vFinal
├── docs/
│   ├── adr/ADR-001..00X  ← decisiones acumulativas
│   ├── arquitectura/     ← C4, contratos, presupuestos
│   ├── diseno/           ← wireframes, usuarios
│   └── seguridad/        ← threat model
└── .github/workflows/    ← CI pipeline
```

CHECKPOINT — RESPONDÉ SIN MIRAR

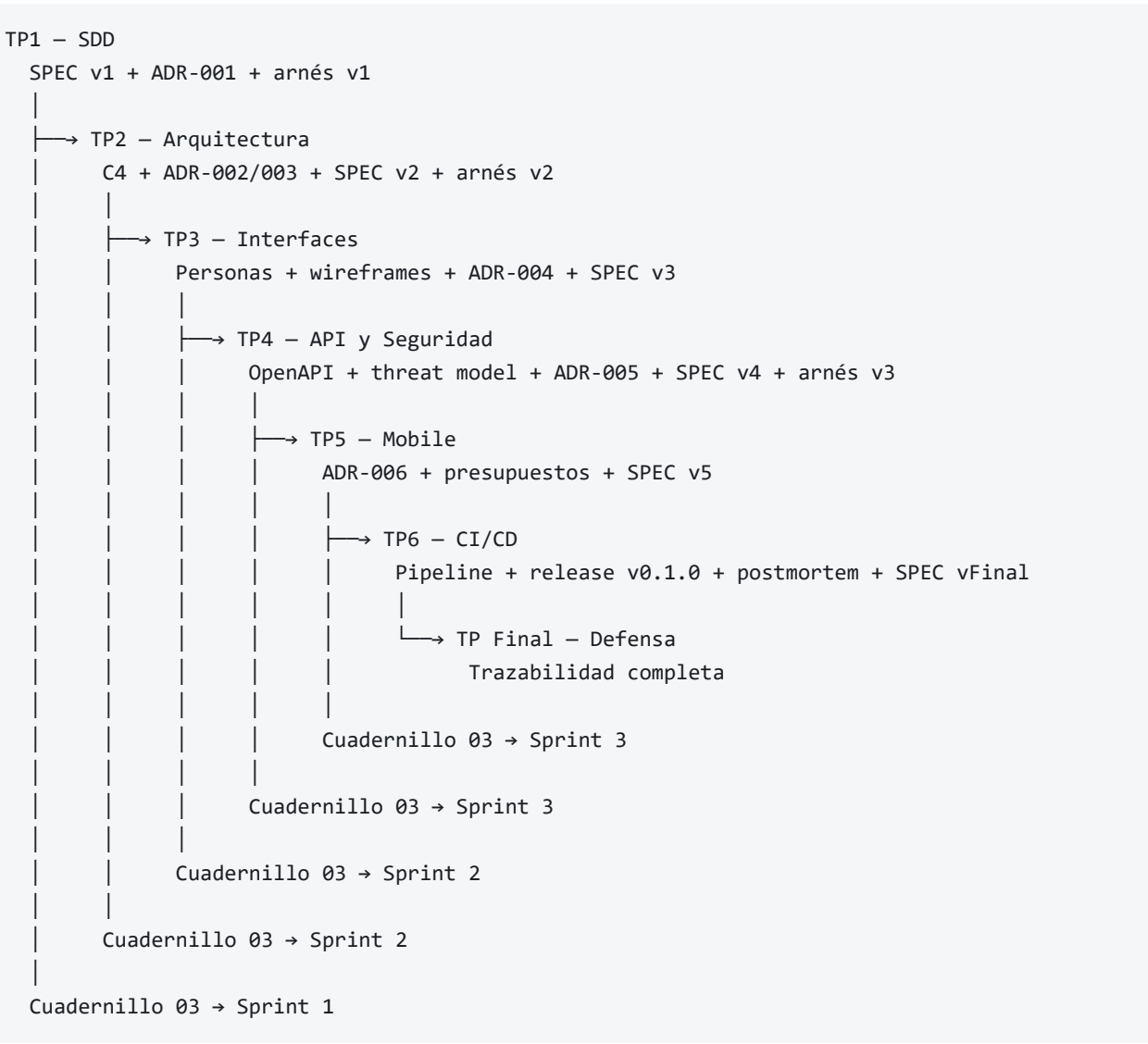
Respondé sin mirar:

1. ¿Qué cuadernillo explica el concepto de ADR y cuál te dice cuándo y cómo crearlo?
2. ¿En qué carpeta de tu repo vive el modelo de amenazas?

Mapa del proyecto

> Los 7 trabajos prácticos forman una cadena: cada uno consume el resultado del anterior y produce la entrada del siguiente.

Cadena de producción



CONCEPTO CLAVE

Trazabilidad: cada decisión, cada diagrama y cada línea de código debe poder rastrearse hasta un requisito de la SPEC. Si alguien pregunta “¿por qué esto es así?”, la respuesta está en los ADR. Si pregunta “¿dónde se verifica?”, la respuesta está en el CI.

Proyecto modelo: A-gobernanza-digital

El repositorio `proyecto-adi-2026/03-proyectos/` contiene un proyecto completo de referencia que podés usar como guía: - SPEC y ADR reales (7 decisiones documentadas) - Diagramas C4 en 3 niveles - Código frontend y backend - Docker Compose para entorno local

No copies su contenido textualmente. Usalo para entender **cómo se ve** un proyecto bien estructurado.

VER EN EL REPO → `03-proyectos/A-gobernanza-digital/README.md`

ERROR FRECUENTE

ERROR FRECUENTE: arrancar el TP2 sin tener el TP1 mergeado. La SPEC es la base de todo; sin ella los ADR no tienen contexto y los diagramas C4 no tienen ancla.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Qué artefacto producido en TP1 necesita TP2 para arrancar?
2. ¿Cuántos ADR acumula un proyecto que completó los 6 TP?
3. ¿Qué cuadernillo explica cómo se evalúa el proyecto?

TP1 — De Vibe Coding a SDD

> **Qué producís:** SPEC v1, ADR-001, arnés v1, repositorio configurado.

Objetivo: migrar tu proyecto a un repositorio individual dentro de la organización `IES9018`, gobernado por especificación formal y arnés de IA.

Entrada

- Repositorio público creado en `IES9018/<tu_nombre>-<tu_proyecto>` (sin fork)

Qué producís

Artefacto	Dónde va	Plantilla
`SPEC.md`	Raíz del repo	`templates/SPEC-template.md`
`docs/adr/ADR-001-stack-tecnologico.md`	`docs/adr/`	`templates/ADR-template.md`
`.opencoderules`	Raíz del repo	Arnés local

SPEC: el contrato del sistema

La SPEC describe propósito, alcance, requisitos funcionales numerados (RF-01, RF-02...), Non-Goals explícitos y contratos de datos. No es un documento de diseño: es contra lo que se mide si el código es correcto.

> CONCEPTO CLAVE
>
> **Non-Goals:** lo que NO hace esta versión. Ej: "esta versión no maneja autenticación de usuarios", "esta versión no genera reportes PDF". Declararlos evita que la IA agregue funcionalidades no pedidas.

ADR-001: decisión de stack

Documentá la elección de tecnologías (lenguaje, framework, base de datos) con al menos dos alternativas descartadas. Cada alternativa debe tener criterios objetivos: curva de aprendizaje, ecosistema, compatibilidad con el proyecto.

> VER CUADERNILLO 01 → ADR

Arnés v1

Creá las primeras reglas que tu agente IA debe respetar: - Alcance permitido (carpetas, tipos de archivo) - Estándares obligatorios (tipado estricto, convención de commits) - Prácticas prohibidas (`any`, dependencias no aprobadas)

SPEC y arnés evolucionan

La SPEC y el arnés NO se quedan quietos después del TP1. Cada TP los actualiza:

TP	SPEC	Arnés
TP1	v1	v1
TP2	v2 (restricciones arquitectónicas)	v2 (coherencia ADR)
TP3	v3 (criterios Gherkin, accesibilidad)	—
TP4	v4 (contratos referenciados)	v3 (seguridad operativa)
TP5	v5 (RNF medibles)	—
TP6	vFinal (changelog completo)	vFinal (consolidado)

EN EL PROYECTO

CUADERNILLO 03: este TP sienta las bases del Sprint 1 de PP3. La SPEC y el arnés son requisitos de auditoría desde el día 1.

VER EN EL REPO → `trabajos-practicos/tp1-sdd-y-arneses/README.md`

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Qué dos artefactos evolucionan a lo largo de todos los TP?
2. ¿Cuál es la diferencia entre un requisito funcional y un Non-Goal?
3. ¿Dónde se guarda el ADR-001 dentro de tu repo?

TP2 — Arquitectura Visible

> **Qué producís:** diagramas C4 (contexto + contenedores), ADR-002, ADR-003, SPEC v2, arnés v2.

Objetivo: hacer visible la arquitectura de tu sistema y registrar las decisiones estructurales.

Entrada

- Especificación inicial (SPEC v1 del TP1)
- Stack tecnológico decidido (ADR-001)

Qué producís

Artefacto	Dónde va	Se entrega
C4-contexto.md	`docs/arquitectura/`	Nivel 1: sistema como caja negra, actores, sistemas externos
C4-contenedores.md	`docs/arquitectura/`	Nivel 2: aplicaciones, bases de datos, protocolos
ADR-002 — Estilo arquitectónico	`docs/adr/`	Elegí entre capas, hexagonal, microservicios
ADR-003 — Persistencia	`docs/adr/`	SQL vs NoSQL vs embebida
SPEC v2	Raíz	Nueva sección: Restricciones arquitectónicas

Diagramas C4

Cada diagrama responde una pregunta distinta. Si no podés escribir la pregunta al pie, el diagrama no cumple.

Nivel	Pregunta
C1 — Contexto	¿Qué sistema estoy construyendo y con quién se relaciona?
C2 — Contenedores	¿Qué apps, servicios y bases de datos lo componen y cómo se comunican?

> VER CUADERNILLO 01 → C4 Model

Regla de oro

Si un elemento no aparece en tu SPEC o en algún ADR, no puede aparecer en el diagrama. El diagrama documenta decisiones tomadas, no deseos.

SPEC v2

Agregá la sección **Restricciones arquitectónicas** citando ADR-001, ADR-002 y ADR-003. Actualizá el changelog (`v1 → v2`, fecha, motivo de cada cambio).

ERROR FRECUENTE

ERROR FRECUENTE: dibujar C4 antes de tener decisiones documentadas. Sin ADR no hay arquitectura que dibujar — tenés deseos, no decisiones.

VER EN EL REPO → `trabajos-practicos/tp2-arquitectura-visible/README.md`

EN EL PROYECTO

CUADERNILLO 03: los diagramas C4 de este TP son los mismos que PP3 te pide verificar en el Sprint 2.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Qué información se ve en un C1 que no se ve en un C2?
2. ¿Por qué no se puede dibujar un elemento en C4 si no aparece en la SPEC o en un ADR?
3. ¿Qué nuevo campo aparece en la SPEC v2 que no estaba en la v1?

TP3 — Interfaces que no Confunden

> **Qué producís:** personas, user journeys, wireframes de baja fidelidad, auditoría heurística, ADR-004, SPEC v3.

Objetivo: definir usuarios reales, sus recorridos críticos y diseñar las pantallas más importantes antes de escribir código.

Entrada

- Arquitectura definida (TP2)
- Restricciones arquitectónicas en SPEC v2

Qué producís

Artefacto	Dónde va	Contenido
Personas + journeys	`docs/diseno/usuarios.md`	2 personas, 2 user journeys en Mermaid
Wireframes	`docs/diseno/wireframes/`	2 pantallas críticas en baja fidelidad
Auditoría heurística	`docs/diseno/auditoria-heuristica.md`	Tabla 10 heurísticas de Nielsen
ADR-004 — Stack UI	`docs/adr/`	Framework de interfaz elegido
SPEC v3	Raíz	Criterios Gherkin, accesibilidad básica

Personas y journeys

Cada persona tiene nombre, rol, objetivo y frustración. No son genéricas: se basan en tu dominio real. Los journeys se modelan en Mermaid como diagramas de secuencia o flowchart.

> VER CUADERNILLO 01 → Heurísticas de usabilidad

Wireframes de baja fidelidad

Las dos pantallas críticas identificadas en los journeys. Cero colores: acá se discute estructura, no estética. Cada wireframe incluye: objetivo, entrada principal, error más probable y cómo lo previene.

Auditoría heurística

Aplicá las 10 heurísticas de Nielsen sobre tus wireframes. Mínimo 3 problemas reales detectados y corregidos. Un hallazgo sin corrección no cuenta.

SPEC v3

Los RF de interfaz ahora incluyen criterios de aceptación en formato **Gherkin** (Given/When/Then). Agregá un requisito de accesibilidad básica: navegación por teclado + contraste AA en las 2 pantallas críticas.

Formato	Ejemplo
Given un usuario no autenticado	
When accede a /turnos	
Then el sistema redirige a /login con código 401	

EN EL PROYECTO

CUADERNILLO 03: los wireframes y la decisión de stack UI son insumos del Sprint 2 de PP3.

VER EN EL REPO → `trabajos-practicos/tp3-diseno-interfaces-hci/README.md`

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuál es la diferencia entre un wireframe y un mockup?
2. ¿Qué significa que un requisito esté en formato Gherkin?
3. ¿Qué heurística de Nielsen se viola si el sistema no muestra qué está pasando después de que el usuario hace click en “Guardar”?

TP4 — Contratos antes que Código

> **Qué producís:** contrato OpenAPI, ADR-005, threat model, arnés v3, SPEC v4.

Objetivo: definir los contratos de tu API antes de implementar, con especificación formal y modelo de amenazas.

Entrada

- Wireframes y decisiones de interfaz (TP3)
- Criterios de aceptación en SPEC v3

Qué producís

Artefacto	Dónde va	Descripción
`api-contracts.yaml`	`docs/arquitectura/`	OpenAPI 3.x, 5 endpoints críticos
ADR-005 — Estrategia web	`docs/adr/`	SPA vs SSR vs MPA vs solo-API
Threat model lite	`docs/seguridad/threat-model-lite.md`	5 amenazas STRIDE con mitigaciones
Arnés v3	Raíz	Reglas de seguridad operativas
SPEC v4	Raíz	Contratos referencian schemas YAML

OpenAPI: contrato primero

Los 5 endpoints críticos de tu SPEC se definen en OpenAPI 3.x antes de escribir código: path, método, parámetros, schemas request/response, códigos de error (400, 401, 404, 500 mínimo). El archivo debe pasar un linter (`@redocly/cli lint`).

> VER CUADERNILLO 01 → OpenAPI

Modelo de amenazas

Aplicá STRIDE a tus 5 endpoints reales. Mínimo 5 amenazas concretas con mitigación referenciada al componente o regla de arnés que la previene.

Amenaza	Mitigación
Inyección SQL	Validación en endpoint + regla de arnés
Autenticación insuficiente	JWT con security scheme en OpenAPI
Exposición de datos sensibles	Filtros de respuesta en endpoint

VER CUADERNILLO 03 → la evaluación de seguridad incluye este modelo de amenazas

ERROR FRECUENTE

ERROR FRECUENTE: implementar la API antes de definir el contrato. Si el contrato no existe, la IA genera endpoints arbitrarios y el frontend consume lo que hay, no lo que se acordó.

VER EN EL REPO → `trabajos-practicos/tp4-arquitectura-web/README.md`

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Qué archivo define los endpoints de tu API y qué formato usa?
2. ¿Qué columnas tiene la tabla del modelo de amenazas?
3. ¿Qué regla nueva agrega el arnés v3 que no estaba en la v2?

TP5 — Estrategia Mobile

> **Qué producís:** ADR-006, presupuestos de rendimiento, estrategia offline, SPEC v5, wireframe adaptativo.

Objetivo: decidir y documentar la estrategia mobile con métricas verificables.

Entrada

- API con contratos definidos (TP4)
- Wireframes de pantallas críticas (TP3)

Qué producís

Artefacto	Dónde va	Descripción
ADR-006 — Estrategia mobile	`docs/adr/`	Matriz de decisión con criterios
Presupuestos de rendimiento	`docs/arquitectura/presupuestos-rendimiento.md`	LCP, INP, peso JS
Estrategia offline	`docs/arquitectura/offline-sync.md`	Offline-first o Non-Goal justificado
Wireframe adaptativo	`docs/diseno/wireframes/`	Pantallas críticas en breakpoint móvil
SPEC v5	Raíz	RNF medibles (RNF-01…)

Matriz de decisión mobile

Compará las opciones (responsive web, PWA, nativa, cross-platform) contra criterios objetivos: costo de mantenimiento, necesidad offline, acceso a hardware, tiempo hasta el cierre.

Criterio	Responsive	PWA	Nativa	Cross-platform
Costo 1 persona	Bajo	Medio	Alto	Medio

Presupuestos de rendimiento

Definí números que puedas medir. Cada presupuesto incluye la herramienta concreta que lo verifica.

Métrica	Presupuesto	Herramienta
LCP (móvil)	< 2.5 s (4G)	Lighthouse CI
INP	< 200 ms	Lighthouse CI
Peso JS inicial	< 200 KB gzip	source-map-explorer

VER CUADERNILLO 01 → Mobile / Offline-First

EN EL PROYECTO

CUADERNILLO 03: el ADR mobile y los presupuestos de rendimiento se evalúan en el Sprint 3 de PP3.

VER EN EL REPO → `trabajos-practicos/tp5-arquitectura-mobile/README.md`

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Qué ejemplo concreto de métrica presupuestada se puede medir en CI?
2. ¿Qué documento registra la decisión de estrategia mobile?
3. Si tu dominio no requiere offline, ¿qué ponés en lugar de la estrategia offline?

TP6 — Integración Continua

> **Qué producís:** pipeline CI, release v0.1.0, arnés vFinal, postmortem, SPEC vFinal.

Objetivo: automatizar las validaciones que antes hacías a mano y cerrar el cuatrimestre con un postmortem honesto.

Entrada

- Los 5 TP anteriores mergeados
- Contrato OpenAPI definido (TP4)
- Presupuestos de rendimiento (TP5)

Qué producís

Artefacto	Dónde va	Descripción
`.github/workflows/ci.yml`	Raíz del repo	Pipeline que lintea, testea y compila
Release v0.1.0	GitHub Releases	Tag anotado + CHANGELOG
Arnés vFinal	Raíz del repo	Consolidado de versiones anteriores
Postmortem	`docs/postmortem-cuatrimestre.md`	3 incidentes reales con IA
SPEC vFinal	Raíz	Changelog v1→vFinal completo

Pipeline CI

Workflow que se ejecuta en cada PR hacia `main`: 1. **Lint del contrato:** valida `api-contracts.yaml` con `@redocly/cli lint` 2. **Tests:** corre la suite unitaria (mínimo 5 tests) 3. **Build:** compila el proyecto

Agregá el badge de estado del CI al README. PR con CI rojo = merge prohibido.

> VER CUADERNILLO 01 → CI/CD y GitHub Actions

Release v0.1.0

`CHANGELOG.md` con formato Keep a Changelog: secciones por categoría (feat, fix, docs) y tag anotado sobre el commit de release.

Postmortem

Tres incidentes reales donde la IA generó código incorrecto. Cada uno con: qué generó mal, cómo se detectó, qué capa lo previene (SPEC, ADR, arnés, CI, revisión humana).

Incidente	Detección	Capa
Endpoint inseguro	Code review	Arnés v3 + threat model
Función que no compila	CI build	Pipeline
Dependencia no autorizada	ADR check	Arnés v2

SPEC vFinal

Estado "congelado para la defensa": changelog completo v1→vFinal y trazabilidad ADR ↔ restricciones vigentes.

> VER EN EL REPO → `trabajos-practicos/tp6-integracion-continua/README.md`

> CHECKPOINT — RESPONDÉ SIN MIRAR
>
> **Respondé sin mirar:**
>
> 1. ¿Qué impide que un PR con CI rojo se mergee?
> 2. ¿Qué tres artefactos produce el TP6 además del pipeline?
> 3. ¿Qué información registra el postmortem?

TP Final — Defensa

> **Qué producís:** recorrido guiado + una decisión con arrepentimiento + evidencia de trazabilidad.

Objetivo: sostener cada decisión de tu repositorio frente a preguntas.

Condiciones de admisión

Requisito	Verificación
TP1–TP6 mergeados	Historial de PRs
CI en verde en `main`	Badge + run del pipeline
Release v0.1.0 etiquetada	Tags del repo
Arnés consolidado final	`.opencoderules`
Postmortem con evidencia	`docs/postmortem-cuatrimestre.md`

Estructura de la defensa

Minuto	Qué hacés
5	Recorrido guiado: `main` → SPEC → ADRs → C4 → CI → release
2	Una decisión con arrepentimiento: elegí un ADR, explicá qué harías distinto
1	Demo de pantalla crítica o Non-Goal justificado
3	Preguntas del docente

> VER CUADERNILLO 03 → los criterios de evaluación detallados están en las rúbricas de PP3

> VER EN EL REPO → `trabajos-practicos/tp-final-integrador/README.md`

> RECORDÁ
>
> **No se necesitan slides.** La defensa es sobre tu repositorio. Si la respuesta está documentada, la encontrás navegando. Si no está documentada, la defensa expone el vacío.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuánto dura la defensa y cómo se distribuye el tiempo?
2. ¿Qué pasa si llegás al TP Final sin tener los TP1–6 mergeados?
3. ¿Qué es una “decisión con arrepentimiento”?

Proyecto Modelo y Plantillas

> Cada TP produce artefactos a partir de plantillas. El proyecto modelo muestra cómo se ve el resultado final.

Proyecto modelo: A-gobernanza-digital

El repositorio `proyecto-adi-2026` contiene un proyecto completo de referencia en `03-proyectos/A-gobernanza-digital/`. No es una plantilla que copies: es una guía de cómo se estructura un proyecto real.

Componente	Ruta en el proyecto modelo	Lo muestra
README del proyecto	`README.md`	Visión general, estructura, cómo correrlo
Especificación	`01_PLAN_MAESTRO/vision.md`	Cómo se ve una visión de proyecto
Casos de uso	`02_CASOS_DE_USO/`	CU-01 a CU-05 con diagramas
Arquitectura	`03_ARQUITECTURA/`	Diagramas C4, ADR-001 a ADR-007, hexagonal
Código fuente	`08_CODIGO_FUENTE/`	Frontend React/TypeScript
Infraestructura	`docker-compose.yml`, `nginx.conf`	Despliegue local

> VER EN EL REPO → `03-proyectos/A-gobernanza-digital/README.md`

Mapa de plantillas

Artefacto	Plantilla	Dónde se guarda en tu repo
SPEC	`templates/SPEC-template.md` → `SPEC.md`	Raíz
ADR	`templates/ADR-template.md` → `docs/adr/ADR-NNN-titulo.md`	`docs/adr/`
PR template	`.github/PULL_REQUEST_TEMPLATE.md`	(automático en GitHub)

ERROR FRECUENTE

ERROR FRECUENTE: tomar el proyecto modelo como plantilla literal y copiar sus decisiones (stack, estructura, ADR) sin adaptarlas a tu dominio. El proyecto modelo muestra **cómo documentar**, no **qué documentar**.

CONCEPTO CLAVE

Plantilla ≠ Ejemplo ≠ Entrega. La plantilla es el formato vacío. El ejemplo es el proyecto modelo completo. La entrega es tu propio contenido siguiendo la plantilla, adaptado a tu dominio.

VER EN EL REPO → `templates/` para SPEC-template.md y ADR-template.md

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuál es la diferencia entre una plantilla y el proyecto modelo?
2. ¿Dónde está la plantilla de SPEC en el repositorio?
3. ¿Cuántos ADR documenta el proyecto modelo A-gobernanza-digital?

Cadena de trazabilidad completa

Cada sección de este cuadernillo mostró un eslabón de la cadena. Juntos forman el ciclo completo:

```
Necesidad de negocio
  ↓
SPEC (TP1) — contrato del sistema
  ↓
ADR (TP1-TP6) — decisiones documentadas
  ↓
C4 (TP2) — arquitectura visible
  ↓
Wireframes (TP3) — interfaz pensada
  ↓
OpenAPI (TP4) — contrato de API
  ↓
Threat model (TP4) — seguridad diseñada
  ↓
Mobile (TP5) — estrategia multiplataforma
  ↓
CI/CD (TP6) — automatización y calidad
  ↓
Defensa (TP Final) — trazabilidad demostrada
```

¿POR QUÉ IMPORTA?

Lo que sigue: tu repositorio individual, con todos los TP mergeados y los artefactos completos, es la entrada del Cuadernillo 03 — PP3, que coordina los sprints, la auditoría y la evaluación final.

Cuadernillo	Te sirve para
01 — Teoría	Entender los conceptos que aplicás acá
03 — PP3	Coordinar cuándo y cómo se evalúa cada artefacto

VER EN EL REPO → `README.md` del proyecto-adi-2026 para el roadmap completo de TP y sus fechas.

RECORDÁ

El repositorio de proyecto (`proyecto-adi-2026`) tiene el desarrollo completo de cada TP, ejercicios guiados, el proyecto modelo y las plantillas. Este cuadernillo te da el mapa de aplicación; el repo es el territorio de las consignas.

CHECKPOINT — RESPONDÉ SIN MIRAR

Revisión final:

1. Revisá los checkpoints de las unidades donde tengas dudas.
2. Verificá que tu repositorio tenga los artefactos que cada unidad menciona.
3. Cuando el docente publique un TP, este cuadernillo te dice qué producir y con qué plantilla.